Generated from `docs/handoff-guides/markdown/design/arch.md` by `tools/guide-publisher`.

Architecture Handoff Guide

Use this guide when you need to describe **system boundaries, parties, bounded contexts, services, and important dependencies**.

This guide is about the **big structural picture**: who owns what, what major parts exist, and how they relate.

KNOWLEDGE AREA

This guide helps with the **big-picture system structure**. Technically, it covers **structural system design and decomposition**.

Guide family map

Where this guide sits in the full Blueprint Handoff Atlas family.

ROOT & CROSS-CUTTING

Handoff Atlas | Blueprint Bundle | Metamodel Vocabulary | Migrations

DESIGN

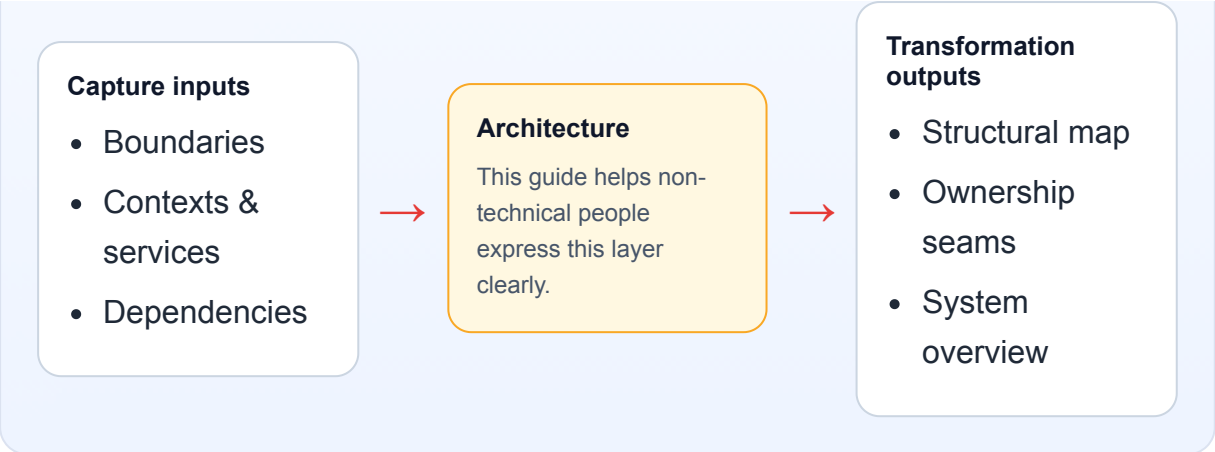
Architecture | Concepts | Domain | Dynamics | Infrastructure | Interactions | Models | Quality | Rules | Story

GOVERNANCE

Capability | Decisions | Motivation | Organization | Roadmap | Leverage | Test Cases | Value Stream

Capture → focus → transformation

What to gather first, what this layer focuses on, and what a modeler or AI agent can produce from it.



Main entities in this guide

- **Party** - a major owner or outside system that hosts one or more architectural areas.
- **Context** - a bounded business area with its own responsibilities, language, and boundaries.
- **Service** - a deployable or logical service inside a context that does work or exposes behavior.
- **Dependency** - a meaningful reliance on another context or external system.
- **Entity** - a core domain thing named inside a context when the structural picture needs that extra detail.
    - Often confused with: **Concept** in the Concepts guide. Use **Entity** here to show structure, not to define business meaning in full.

What belongs here

- major systems or organizational parties
- bounded contexts or domain boundaries
- services or major components
- dependencies between major parts
- ownership and responsibility boundaries

What does not belong here

- detailed business process steps
- detailed screen behavior
- low-level infrastructure configuration

Core things to capture

- the main parties involved
- the contexts or major domains they own
- the services or major components that matter
- the purpose of each major part
- key dependencies and interfaces between parts

Core relationships to capture

- party -> context
- context -> service/component
- service -> dependency on another service/context
- owner -> owned structural element

Modus operandi

Start broad, then refine:

1. identify the big parties or systems
2. identify the main contexts they own
3. identify the important services/components inside those contexts
4. describe the most meaningful dependencies
5. stop before you fall into implementation detail

Prompt set

- what are the main system or business boundaries?
- who owns each major area?
- what major services/components exist?
- what depends on what?
- where are the most important interfaces or seams?

Free-text intake template

- **Architecture scope:**
- **Main parties/systems:**
- **Contexts/domains:**
- **Key services/components per context:**
- **Important dependencies:**
- **Ownership notes:**
- **Open structural concerns:**

Worked example

- Commerce Platform party owns Storefront and Post-Purchase contexts
- Post-Purchase context contains Return Service and Refund Service
- Return Service depends on Notification Service for status updates
- Reporting context consumes data from both Checkout and Post-Purchase

Layer-specific handoff guidance

For architecture, the handoff should help the receiver reconstruct the **structural picture** fast.

Make sure the handoff clearly states:

- the major boundaries
- the parties or owners of those boundaries
- the important dependencies between them

If the receiver cannot redraw the high-level structure from your notes, add clearer boundary and dependency statements.

How this becomes YAML

A technical modeler or AI agent can transform this into architecture structures by mapping parties, contexts, services, and dependencies into a navigable structural view.

Common mistakes

- listing every technical component instead of the meaningful structure
- mixing process flow with architecture boundaries
- naming dependencies without explaining why they exist
- omitting ownership